Centro Universitário de Excelência
Sistemas de Informação

MecâniQA – OAT 3

Autores: Érica Araujo
Mickael Cedraz
Monyc Luisa
Nalbert Santana
Pedro César
Rafael Santos

Feira de Santana, 2026

Agenda

O objetivo dessa apresentação é mostrar como treinamos, otimizamos e comparamos modelos de regressão para reduzir os erros nas previsões de trocas de óleo.

1. Contexto:
O problema e a necessidade da MecâniQA.

2. Modelos Iniciais:
Dados, atributos e validação temporal.

3. Grid Search:
Busca dos melhores hiperparâmetros.

4. Random search:
Tempo computacional x Precisão.

5. Resultados:
Métricas, gráfico e lições aprendidas.

CONTEXTO

O desafio é transformar histórico diário em previsões mais úteis.

A EMPRESA

A MecâniQA apoia oficinas e auto centers de Feira de Santana e região.

O PROBLEMA

Picos e quedas na demanda dificultam peças, estoque e equipes.

A META

Ir além dos baselines e encontrar o melhor equilíbrio entre erro e processamento.

PREPARAÇÃO DOS DADOS

O modelo aprende a partir de informações do passado, nunca do próprio futuro.

```python
dados = df.sort_index().copy()
dados["Lag_1"] = dados["Trocas_Oleo"].shift(1)
dados["Lag_7"] = dados["Trocas_Oleo"].shift(7)
dados["Media_7"] = dados["Trocas_Oleo"].shift(1).rolli
dados["Dia_Semana"] = dados.index.dayofweek
colunas = ["Lag_1", "Lag_7", "Media_7", "Dia_Semana"]
dados = dados.dropna(subset=colunas + ["Trocas_Oleo"])

X = dados[colunas]
y = dados["Trocas_Oleo"]
```

✓ 0.0s Python

Lag 1	Demanda do dia anterior
Lag 7	Demanda de uma semana atrás
Média 7	Média dos sete dias anteriores
Dia da semana	Posição na rotina semanal

MODELOS DE MACHINE LEARNING

Árvore de Decisão e Random Forest com parâmetros padrão do scikit-learn.

```
modelos = {
    "Árvore de Decisão": DecisionTreeRegressor(random_state=42),
    "Random Forest": RandomForestRegressor(random_state=42)
}
```

```
for nome, modelo in modelos.items():

    modelo.fit(X_treino, y_treino)
    previsoes[nome] = modelo.predict(X_teste)
```

- Regressão estima uma quantidade, como trocas de óleo.
- Árvore de Decisão aprende regras a partir dos dados.
- Árvore de Decisão aprende regras a partir dos dados.

MODELOS INICIAIS X BASELINE

Modelo	RMSE	MAPE (%)
Random Forest	6.48	24.67
Árvore de Decisão	6.57	24.23
Baseline Média 7	8.10	37.39

- Baseline: previsão simples usada como referência.
- RMSE mede o erro e pesa mais os desvios grandes.
- Os regressores reduziram o RMSE no teste temporal.

RANDOM SEARCH

```
random = RandomizedSearchCV(
    modelo,
    parametros,
    n_iter=10,
    scoring="neg_mean_absolute_error",
    cv=tscv,random_state=42, n_jobs=-1
)

inicio = time.perf_counter()
random.fit(X, y)
tempo_random = time.perf_counter() - inicio
```

- Sorteia apenas parte das combinações possíveis.
- n_iter=10 limita a busca a dez testes de configuração.
- time.perf_counter() registra o tempo de execução.

GRID SEARCH

```
grid = GridSearchCV(
    modelo, parametros, scoring="neg_mean_absolute_error",
    cv=tscv, n_jobs=-1
)
inicio = time.perf_counter()
grid.fit(X, y)
tempo = time.perf_counter() - inicio

mae_tunado = -grid.best_score_
reducao = (mae_padrao - mae_tunado) / mae_padrao * 100

print(f"\n{nome}")
print("Melhores parâmetros:", grid.best_params_)
```

ÁRVORE DE DECISÃO	RANDOM FORESTE
MAE padrão: 5.40	MAE padrão: 4.73
MAE tunado: 4.75	MAE tunado: 4.44
Redução do MAE: 11.92%	Redução do MAE: 6.03%

TEMPO X ERRO

	Modelo	Método	MAE	Tempo (s)	Melhores parâmetros
0	Random Forest	Grid Search	4.44	175.06	{'max_depth': 10, 'min_samples_leaf': 2, 'min_...
1	Random Forest	Random Search	4.50	6.84	{'n_estimators': 50, 'min_samples_split': 5, '...
2	Árvore de Decisão	Grid Search	4.75	54.94	{'max_depth': 5, 'min_samples_leaf': 1, 'min_s...
3	Árvore de Decisão	Random Search	5.07	0.69	{'min_samples_split': 5, 'min_samples_leaf': 4...

No Random Forest, Random Search foi 96,1% mais rápido, com MAE 4,50 versus 4,44.

COMO AVALIAMOS OS MODELOS

```
def calcular_metricas(real, previsto):
    real = np.asarray(real, dtype=float)
    previsto = np.asarray(previsto, dtype=float)
    mae = mean_absolute_error(real, previsto)
    rmse = np.sqrt(mean_squared_error(real, previsto))
    validos = real != 0
    mape = mean_absolute_percentage_error
    (real[validos], previsto[validos]
     ) * 100 if validos.any() else np.nan
    return {"MAE": mae, "RMSE": rmse, "MAPE (%)": mape}
```

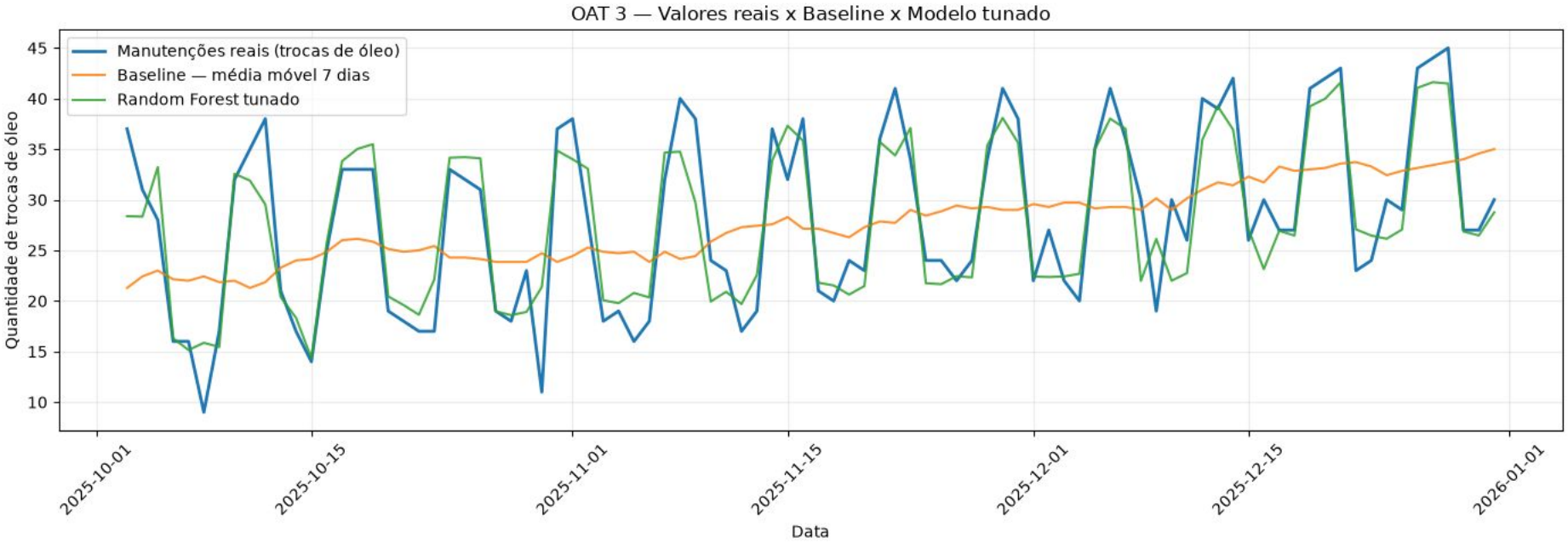
- MAE: erro médio em número de trocas de óleo.
- RMSE: dá maior peso aos erros mais
- altos.
- MAPE: transforma o erro em percentual.

COMPARAÇÃO FINAL

Modelo	MAE	RMSE	MAPE (%)
Baseline Média 7	7.39	8.16	31.13
Random Forest Inicial	3.04	3.98	13.64
Random Forest Tunado	3.16	4.27	14.37

```
MAE: melhoria do tunado vs. baseline = +57.21% | vs. inicial = -3.88%
RMSE: melhoria do tunado vs. baseline = +47.61% | vs. inicial = -7.29%
MAPE (%): melhoria do tunado vs. baseline = +53.82% | vs. inicial = -5.37%
Atenção: porcentagens negativas significam piora de desempenho, não ganho.
```

PREVISÕES X DEMANDA REAL



MecâniQA - OAT 3

Considerações finais

- Os modelos de regressão superaram o baseline de média móvel no conjunto de teste.
- O Random Search reduziu bastante o tempo, com pequena diferença no MAE.
- O Random Forest inicial teve erro ligeiramente menor que o tunado na avaliação final.

REFERÊNCIAS

BELÉM (Equipe). Notebooks dos encontros 1 a 4 e análise de trade-off. MecâniQA, 2026.

MECÂNIQA AUTOMOTIVE TECH. ***OAT 3 — Tunando o Meu Modelo.*** Programa de Trainee 2026.2. 2026.

UNEX. Caderno de Guias — OAT 3: Tutoriais. Modelos de Aprendizagem de Máquina, 2026.

SCIKIT-LEARN. ***Model selection: TimeSeriesSplit, GridSearchCV e RandomizedSearchCV.*** Disponível em: scikit-learn.org.

SCIKIT-LEARN. ***Metrics and scoring***. Disponível em: scikit-learn.org.

BELÉM (Equipe). ***mecaniQA-belem. GitHub***, 2026. Disponível em: github.com/ericaaraujoj/mecaniQA-belem.